

Python on the IBMi – Chapter 3



Our adventurer now finds himself at the edge of a vast canyon, one that separates the modern realm of Python from the ancient stronghold of RPG. With no clear bridge in sight, he grips the rope, takes a breath, and swings across the chasm, powered by *pyodbc* and the promise of discovery.

In this chapter, we'll attempt a daring maneuver: using *pyodbc* to call an external stored procedure that, in turn, invokes an RPG program buried deep within the IBMi. The goal is to pass a message into the void and receive a returned value, a signal that the connection has been made, and the old magic still lives.

The drop below is steep, and the code must be precise. But fortune favors the bold. And on the other side of the canyon, new knowledge awaits. Grab a rope and hold on tight.

Prerequisites

- Please refer to chapter 1 and 2 for other prerequisites
(<https://www.linkedin.com/pulse/python-ibmi-chapter-1-ricky-thompson-zg7te>)
(<https://www.linkedin.com/pulse/python-ibmi-chapter-2-ricky-thompson-sulze/>)
- Code can be found here: (https://github.com/thomprl/ibmi_python_rpg_adventures)
- *pyodbc* - Python ODBC Library (<https://github.com/mkleehammer/pyodbc/wiki>)

Objective

- Call a RPG program from a Python script using pyodbc.

Let's Code

Calling a RPG program using pyodbc proved to be more of a challenge than I first thought it would be. I had to enlist some help with this one from the gurus over at (<https://chat.ibmioss.org/>). Let me say if you aren't part of this group sign-up. A lot of good information gets passed around. This site is considered IBMi Open Source Chat Group but it's so much more. Korinne Adler showed me how to call an external stored procedure from pyodbc. If I really needed to call an RPG program I think the better solution would be to stick with iToolkit from the previous article. But it's always good to have a couple of tools in your toolbelt.

So from the previous article we created a RPG program called EXAMPLE1. Now we are going to create an external stored procedure to call that RPG program. I ran the following SQL in VS Code in the DB2 for i extension to create the external stored procedure.

```
DROP PROCEDURE RTHOMPSON1/EXAMPLE1;
CREATE PROCEDURE RTHOMPSON1/EXAMPLE1 (
  IN p_InParm  CHAR(20),
  OUT p_OutParm CHAR(200))
LANGUAGE RPGLE
SPECIFIC RTHOMPSON1/EXAMPLE1
NOT DETERMINISTIC
CALLED ON NULL INPUT
EXTERNAL NAME RTHOMPSON1/EXAMPLE1
PARAMETER STYLE SQL ;
```

Now lets test the external stored procedure to make sure it's working.

NOTE: There is a bug as of right now in the VS Code DB2 for i extension that will not return output parameters from stored procedures. Here is a work around but hopefully this will be fixed in the future. You have to put the output parameter in a global variable.

```
CREATE OR REPLACE VARIABLE RTHOMPSON1.OUT_PARM CHAR(200) CCSID
1208 DEFAULT '';
call rthompson1.example1 ('IBMi', RTHOMPSON1.OUT_PARM);
values (RTHOMPSON1.OUT_PARM);
```

The results should look like the following in VS Code

```
15 CREATE OR REPLACE VARIABLE RTHOMPSON1.OUT_PARM CHAR(200) CCSID 1208 DEFAULT '';
16 call rthompson1.example1 ('IBMi', RTHOMPSON1.OUT_PARM);
17 values (RTHOMPSON1.OUT_PARM);
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS IBMI IBMI JOB LOG COMMENTS

✓ RESULTS

00001

Welcome IBMi, to Python/RPGLE programming! - Job Name: 10216/QUSER/QZDASOINIT

Loaded 1 rows in 243ms. End of data.

From an ACS Run SQL Scripts session you can test the stored procedure by running the following:

```
cl: addlib rthompson1;
call rthompson1.example1 ('IBMi', '');
```

The results should look like the following in ACS RSS.

File Edit Search View Connection Run Explain Monitor Editor Tools Help

*Untitled 1

```
1 cl: addlib rthompson1;
2 call rthompson1.example1 ('IBMi', '');
3
```

[07/15/2025, 09:23:55 PM] Run Selected...

call rthompson1.example1 ('IBMi', '')

✓ Return Code = 0

Output Parameter #2 (P_OUTPARM) = Welcome IBMi, to Python/RPGLE programming! - Job Name: 10243/QUSER/QZDASOINIT

✓ Statement ran successfully (263 ms)

Calling the RPG from Python using pyodbc

In my (/home/rthompson/python/chapter3) folder I created a new Python script called pyodbc_callrpg.py with the following code.

```
import pyodbc
connstring = 'DSN=*LOCAL;CommitMode=0;'
try:
    conn = pyodbc.connect(connstring)
```

```

print("Connected to the DB2")

# Create a variable to hold the output parameter
conn.execute("CREATE OR REPLACE VARIABLE RTHOMPSON1.OUT_PARM
CHAR(200) CCSID 1208 DEFAULT ''")
# Set the output parameter value to the job name of the
current job
conn.execute("SET RTHOMPSON1.OUT_PARM = qsys2.job_name")
# Fetch the value in our variable to get the job name
running this Python Program
# This can be handy if needing to debug the program
data =
conn.execute("VALUES (RTHOMPSON1.OUT_PARM)").fetchval()

# Print the job name
print(f"Output Parameter Value: {data}")

# Reset the output parameter to an empty string before
calling the stored procedure
conn.execute("SET RTHOMPSON1.OUT_PARM = ''")

# Define the stored procedure call
sql = "{CALL RTHOMPSON1.EXAMPLE1(?, RTHOMPSON1.OUT_PARM)}"
# Input parameter value
in_parm = "IBMi"
# Define the parameters
parms = (in_parm)

# Bind parameters
conn.execute(sql, parms)

# Fetch the output parameter value
data =
conn.execute("VALUES (RTHOMPSON1.OUT_PARM)").fetchval()
# Print the output parameter value
print(f"Output Parameter Value: {data}")

except pyodbc.Error as e:
    print(f"Error: {e}")

finally:
    # Close cursor and connection
    conn.close()

```

I wanted to highlight the code below. The question mark (?) is a place holder for the variable we are passing in. Pyodbc doesn't have a way to retrieve an output parameter from a call statement so we must place it in our variable RTHOMPSON1.OUT_PARM so that we can retrieve it later.

```
# Define the stored procedure call
sql = "{CALL RTHOMPSON1.EXAMPLE1(?, RTHOMPSON1.OUT_PARM) }"
# Input parameter value
in_parm = "IBMi"
# Define the parameters
parms = (in_parm)

# Bind parameters
conn.execute(sql, parms)
```

From VS Code Open a PASE terminal by pressing (**Ctrl+Shift+J**) and selecting PASE and then type the following:

```
cd python/chapter3
python3 pyodbc_callrpg.py
```

If successful your results should look like the following.

```
-bash-5.2$ python3 pyodbc_callrpg.py
Connected to the DB2
Output Parameter Value: 798570/QUSER/QZDASOINIT

Output Parameter Value: Welcome IBMi, to Python/RPGLE programming! - Job Name: 79857/QUSER/QZDASOINIT
-bash-5.2$ █
```

BONUS TIP

If you wanted to debug this (like I had to do). You could put a delay job in the EXAMPLE1 RPG program. Then call the Python program it will output the job name and then call the RPG program. This would give you enough time to do a STRSRVJOB and STRDBG to debug the program using the job name from the Python output.

What's Next?

Just when our hero thought the path was clear, he stumbles into a sticky trap. Deep within the shadows of the web, a monstrous spider descends, it's tangled threads representing new challenges in serving data across the network.

In Chapter 4, we'll explore how to spin up a simple Python web server on the IBMi. It's not all magic, but when it works, it sure feels like it.

So grab your shield, your courage, and your browser, our journey into the web begins next.